

Control Flow in R & Python

Lecture 03

Dr. Colin Rundel

Logical & comparison operators

Comparison operators

The syntax is nearly identical. The key difference is that R's comparison operators are *vectorized* (element-wise, returning a logical vector), whereas Python compares two objects and returns a single `bool`.

Comparison	R	Python
less than	<code>x < y</code>	<code>x < y</code>
greater than	<code>x > y</code>	<code>x > y</code>
less than or equal to	<code>x <= y</code>	<code>x <= y</code>
greater than or equal to	<code>x >= y</code>	<code>x >= y</code>
equal to	<code>x == y</code>	<code>x == y</code>
not equal to	<code>x != y</code>	<code>x != y</code>
membership	<code>x %in% y</code>	<code>x in y</code>

Comparisons

R compares element by element, while Python compares the objects as a whole and uses `in` for membership.

```
1 x = c("A", "B", "C")
```

R

```
1 x == "A"
```

R

```
[1] TRUE FALSE FALSE
```

```
1 "A" %in% x
```

R

```
[1] TRUE
```

```
1 c(1, 2, 3) == c(1, 2, 3)
```

R

```
[1] TRUE TRUE TRUE
```

```
1 c(1, 2, 3) < c(1, 3, 0)
```

R

```
[1] FALSE TRUE FALSE
```

```
1 x = ["A", "B", "C"]
```

py

```
1 x == "A"
```

py

```
False
```

```
1 "A" in x
```

py

```
True
```

```
1 [1, 2, 3] == [1, 2, 3]
```

py

```
True
```

```
1 [1, 2, 3] < [1, 3, 0]
```

py

```
True
```

Python's ordered comparisons of lists are *lexicographic* - more on this in a moment, and on recycling and vectorization

Comparing strings

Both languages can order strings, but differently - Python compares by Unicode code point (so ASCII **A-Z** sort before ASCII **a-z**), while R uses the collation rules of the current locale (roughly alphabetical, with case as a tie breaker).

```
1 "A" < "B" R
```

[1] TRUE

```
1 "Good" < "Goodbye" R
```

[1] TRUE

```
1 "A" < "a" R
```

[1] FALSE

```
1 "a" < "B" R
```

[1] TRUE

```
1 "Z" < "a" R
```

[1] FALSE

```
1 "A" < "B" py
```

True

```
1 "Good" < "Goodbye" py
```

True

```
1 "A" < "a" py
```

True

```
1 "a" < "B" py
```

False

```
1 "Z" < "a" py
```

True

See `Sys.getlocale("LC_COLLATE")` and `?Comparison` for R's locale (in the **C** locale R also orders by code point), and `locale.strcoll() / locale.strxfrm()` for locale-aware ordering in Python. The same rules apply to `sort()` and

Lexicographic ordering

Ordered comparisons of strings, lists, and tuples in Python, are *lexicographic* (dictionary order) - elements are compared pairwise from the front and the first difference decides the result. If one sequence runs out first, it sorts before the longer one.

```
1 "card" < "cart" R
```

[1] TRUE

```
1 "cat" < "card" R
```

[1] FALSE

```
1 "car" < "card" R
```

[1] TRUE

```
1 "card" < "cart" py
```

True

```
1 "cat" < "card" py
```

False

```
1 "car" < "card" py
```

True

```
1 [1, 2, 3] < [1, 3, 0] py
```

True

```
1 (1, 2) < (1, 2, 5) py
```

True

Only the elements up to the first difference are ever compared. `[0, "a"] < [1, 2]` is `True`, but `[1, "a"] < [1, 2]`

Logical operators

Operation	R (vectorized)	R (scalar)	Python
and	<code>x & y</code>	<code>x && y</code>	<code>x and y</code>
or	<code>x y</code>	<code>x y</code>	<code>x or y</code>
not	<code>!x</code>		<code>not x</code>
exclusive or	<code>xor(x, y)</code>		<code>x != y</code>

Python has `&`, `|`, and `~` operators, but they are *bitwise* operators for integers; confusingly, `numpy` and `pandas` overload them as element-wise logical operators (more on this later).

Vectorized vs scalar

In R you choose between the vectorized and scalar forms; in base Python `and` / `or` are always scalar, though with lists they can misleadingly look vectorized.

```
1 x = c(TRUE, FALSE, TRUE)
2 y = c(FALSE, TRUE, TRUE)
```

R

```
1 x = [True, False, True]
2 y = [False, True, True]
```

py

```
1 x | y
```

R

```
1 x or y # returns x
```

py

```
[1] TRUE TRUE TRUE
```

```
[True, False, True]
```

```
1 x & y
```

R

```
1 x and y # returns y
```

py

```
[1] FALSE FALSE TRUE
```

```
[False, True, True]
```

```
1 x || y
```

R

```
Error in `x || y`:  
! 'length = 3' in coercion to 'logical(1)'
```

Since R 4.3, the `&&` and `||` operators throw an error if either side has length greater than 1 (older versions silently used

Short-circuit evaluation

`&&` / `||` in R and `and` / `or` in Python evaluate their left operand first and only evaluate the right operand if the left one does not already decide the outcome.

```
1 FALSE && stop("Error")
```

R

```
[1] FALSE
```

```
1 TRUE || stop("Error")
```

R

```
[1] TRUE
```

```
1 TRUE && stop("Error")
```

R

```
Error:
! Error
```

```
1 FALSE || stop("Error")
```

R

```
Error:
! Error
```

```
1 False and 1/0
```

py

```
False
```

```
1 True or 1/0
```

py

```
True
```

```
1 True and 1/0
```

py

```
ZeroDivisionError: division by zero
```

```
1 False or 1/0
```

py

```
ZeroDivisionError: division by zero
```

Guarding with short-circuits

This makes them useful as *guards*, where an earlier condition rules out inputs that would cause a later condition to error.

```
1 x = "abc"
2 abs(x) > 1
```

R

Error in `abs()`:
! non-numeric argument to mathematical function

```
1 is.numeric(x) && abs(x) > 1
```

R

[1] FALSE

```
1 x = NULL
2 abs(x) > 1
```

R

Error in `abs()`:
! non-numeric argument to mathematical function

```
1 !is.null(x) && abs(x) > 1
```

R

[1] FALSE

```
1 x = "abc"
2 abs(x) > 1
```

py

TypeError: bad operand type for abs(): 'str'

```
1 isinstance(x, (int, float)) and abs(x) > 1
```

py

False

```
1 x = None
2 abs(x) > 1
```

py

TypeError: bad operand type for abs(): 'NoneType'

```
1 x is not None and abs(x) > 1
```

py

False

Truthiness and short-circuiting

Python's `and` and `or` accept any values, not just `bool`s. They check the *truthiness* of `x` and short-circuiting decides which value comes back:

- `x and y` - if `x` is falsy the result is `x`, otherwise it is `y`
- `x or y` - if `x` is truthy the result is `x`, otherwise it is `y`

Either way the result is one of the original values, unchanged, whereas R's `&&` and `||` always produce a single `TRUE`, `FALSE`, or `NA`.

```
1 1 and 2 # 1 is truthy -> 2
```

py

2

```
1 0 and 2 # 0 is falsy -> 0
```

py

0

```
1 [] and [1] # [] is falsy -> []
```

py

[]

```
1 0 or "default" # 0 is falsy -> "default"
```

py

'default'

```
1 "" or "default" # "" is falsy -> "default"
```

py

'default'

```
1 "abc" or "default" # "abc" is truthy -> "abc"
```

py

'abc'

Fallback values

```
1 x or default
```

py

is a common Python idiom for supplying a default when `x` is falsy. R 4.4 added the null coalescing operator `%||%` for the narrower case where `x` is `NULL`.

```
1 name = ""
2 name or "anonymous"
```

py

'anonymous'

```
1 name = "Colin"
2 name or "anonymous"
```

py

'Colin'

```
1 name = NULL
2 name %||% "anonymous"
```

R

[1] "anonymous"

```
1 name = "Colin"
2 name %||% "anonymous"
```

R

[1] "Colin"

Conditionals

if and else

```
1 x = 3
2 if (x > 0) {
3   print("x is positive")
4 }
```

R

[1] "x is positive"

```
1 if (x > 0) {
2   print("x is positive")
3 } else {
4   print("x is not positive")
5 }
```

R

[1] "x is positive"

```
1 x = 3
2 if x > 0:
3   print("x is positive")
```

py

x is positive

```
1 if x > 0:
2   print("x is positive")
3 else:
4   print("x is not positive")
```

py

x is positive

R wraps the condition in `()` and the body in `{}`, while Python ends the condition with `:` and the body is the following *indented* block.

In R the `{}` are optional for a single-expression body, e.g. `if (x > 0) print("x is positive")`, but the we and the

else if and elif

Conditions are checked in order and only the *first* true branch runs; the optional `else` branch runs if no other branch triggered.

```
1 x = 0
2 if (x < 0) {
3     print("x is negative")
4 } else if (x > 0) {
5     print("x is positive")
6 } else {
7     print("x is zero")
8 }
```

R

```
[1] "x is zero"
```

```
1 x = 0
2 if x < 0:
3     print("x is negative")
4 elif x > 0:
5     print("x is positive")
6 else:
7     print("x is zero")
```

py

```
x is zero
```

Conditionals as expressions

R's `if` is an expression that returns the value of the evaluated branch, so it can be used directly in an assignment. Python's `if` is a *statement* that returns nothing; instead there is a separate *conditional expression*, `a if cond else b`.

```
1 x = 5
2 s = if (x %% 2 == 0) x / 2 else 3 * x + 1
3 s
```

[1] 16

```
1 x = 5
2 s = x / 2 if x % 2 == 0 else 3 * x + 1
3 s
```

16

Both are equivalent to assigning within each branch,

```
1 if (x %% 2 == 0) {
2   s = x / 2
3 } else {
4   s = 3 * x + 1
5 }
6 s
```

[1] 16

```
1 if x % 2 == 0:
2     s = x / 2
3 else:
4     s = 3 * x + 1
5
6 s
```

16

Conditionals are *not* vectorized

```
1 x = c(1, 3)
```

R

```
1 if (x == 1) print("x is 1!")
```

R

Error in `if (x == 1) ...`:
! the condition has length > 1

```
1 if (x == 3) print("x is 3!")
```

R

Error in `if (x == 3) ...`:
! the condition has length > 1

```
1 x = [1, 3]
```

py

```
1 if x == 1:  
2     print("x is 1!")  
3 else:  
4     print("x is not 1!")
```

py

x is not 1!

```
1 if [False, False]:  
2     print("non-empty lists are truthy")
```

py

non-empty lists are truthy

Since R 4.2, `if` throws an error if the condition has length > 1 (older versions used the first value with a warning).

Python does not automatically apply the comparison element-wise - `x == 1` is simply `False`, and any non-empty list is truthy regardless of its contents.

Collapsing logical vectors

Both languages provide `any()` and `all()` for reducing multiple logical values to a single one.

```
1 x = c(3, 4, 1)
2 x >= 2
```

R

```
[1] TRUE TRUE FALSE
```

```
1 any(x >= 2)
```

R

```
[1] TRUE
```

```
1 all(x >= 2)
```

R

```
[1] FALSE
```

```
1 if (any(x == 3)) print("x contains 3!")
```

R

```
[1] "x contains 3!"
```

```
1 x = [3, 4, 1]
2 any([True, True, False])
```

py

```
True
```

```
1 all([True, True, False])
```

py

```
False
```

```
1 if 3 in x: print("x contains 3!")
```

py

```
x contains 3!
```

Conditionals and truthiness

Python's conditionals accept any object and use its *truthiness*. R's `if` requires a single logical value, though it will coerce other values that `as.logical()` recognizes.

```
1 if (1) "yes" else "no"
```

R

```
[1] "yes"
```

```
1 if (0) "yes" else "no"
```

R

```
[1] "no"
```

```
1 if ("TRUE") "yes" else "no"
```

R

```
[1] "yes"
```

```
1 if ("abc") "yes" else "no"
```

R

```
Error in `if` ("abc") ...`:  
! argument is not interpretable as logical
```

```
1 if (NULL) "yes" else "no"
```

R

```
Error in `if` (NULL) ...`:  
! argument is of length zero
```

```
1 "yes" if 1 else "no"
```

py

```
'yes'
```

```
1 "yes" if 0 else "no"
```

py

```
'no'
```

```
1 "yes" if "abc" else "no"
```

py

```
'yes'
```

```
1 "yes" if [] else "no"
```

py

```
'no'
```

```
1 "yes" if None else "no"
```

py

```
'no'
```

Vectorized conditionals

R's `ifelse()` is the element-wise counterpart to `if` - given a logical vector it returns a vector of the same length built from the `yes` and `no` arguments.

```
1 x = c(-2, 0, 3)
2 ifelse(x > 0, "positive", "non-positive")
```

R

```
[1] "non-positive" "non-positive" "positive"
```

```
1 ifelse(x > 0, x, -x)
```

R

```
[1] 2 0 3
```

Base Python has no equivalent; the idiomatic approach is a list comprehension with a conditional expression (next time).

Conditionals and missing values

- R - `NA` is sticky in comparisons and `if` errors on `NA` rather than guessing. `any()` and `all()` follow the `|` and `&` rules.
- Python - `None` supports equality tests, but ordering it with a number raises `TypeError`. For `nan`, ordered comparisons and `==` are `False`; `!=` is `True`; and despite comparing equal to nothing, `nan` is truthy.

Check for missing values directly with `is.na()`, `x is None`, or `math.isnan()`.

```
1 x = NA
2 x > 1
```

R

[1] NA

```
1 if (x > 1) print("x is big")
```

R

Error in `if (x > 1) ...`:
! missing value where TRUE/FALSE needed

```
1 any(c(1, NA, 4) >= 3)
```

R

[1] TRUE

```
1 all(c(1, NA, 4) >= 1)
```

R

[1] NA

```
1 x = None
```

py

```
1 if x > 1: print("x is big")
```

py

TypeError: '>' not supported between instances of 'None'

```
1 if x is None: print("x is missing")
```

py

x is missing

```
1 (math.nan > 1, math.nan == math.nan, math.nan != 1)
```

py

(False, False, True)

```
1 bool(math.nan)
```

py

True

Multi-way branching

R's `switch()` selects a branch based on a character (or integer) value, while Python 3.10+ has the `match` statement (which also supports much more general *structural pattern matching*).

```
1 x = "b"
2 switch(x,
3     a = "apple",
4     b = "banana",
5     "unknown"
6 )
```

R

[1] "banana"

```
1 x = "b"
2 match x:
3     case "a":
4         print("apple")
5     case "b":
6         print("banana")
7     case _:
8         print("unknown")
```

py

banana

`switch()` uses an unnamed final argument as the default (without one an unmatched value returns `NULL`, invisibly), `case`

Errors

Signaling conditions

R has several ways of communicating with the user beyond `print()` / `cat()`, each of which is a *condition* that can be handled programmatically:

- `message()` - diagnostic messages (sent to stderr)
- `warning()` - something unexpected but not fatal, execution continues
- `stop()` - an error, execution halts

Python has rough equivalents in `print()` (or the `logging` module), `warnings.warn()`, and `raise` with an *exception* object.

```
1 message("Starting")
```

R

Starting

```
1 warning("Something unexpected")
```

R

Warning: Something unexpected

```
1 import warnings
2 print("Starting")
```

py

Starting

```
1 warnings.warn("Something unexpected")
```

py

<string>:1: UserWarning: Something unexpected

Errors stop a Quarto document from rendering unless the chunk sets `#| error: true` (which is how these slides show

Raising errors

```
1 x = -1
2 if (x < 0) {
3   stop("x must be non-negative")
4 }
```

R

Error:

! x must be non-negative

```
1 x = -1
2 if x < 0:
3   raise ValueError("x must be non-negative")
```

py

ValueError: x must be non-negative

Both languages also have a shorthand for checking assumptions - R's `stopifnot()` and Python's `assert` statement:

```
1 stopifnot(x >= 0)
```

R

Error:

! x >= 0 is not TRUE

```
1 stopifnot("x must be non-negative")
```

R

Error:

! x must be non-negative

```
1 assert x >= 0
```

py

AssertionError

```
1 assert x >= 0, "x must be non-negative"
```

py

AssertionError: x must be non-negative

Python's `assert` statements are stripped when running with `python -O`, so use them for development-time checks

Errors vs exceptions

Python exceptions are objects with a class hierarchy - the class describes what went wrong and allows handling to be selective. R errors are, by default, all of the same class (`simpleError`) and are distinguished only by their message.

```
1 "abc" + 1
```

R

```
Error in `"abc" + 1`:  
! non-numeric argument to binary operator
```

```
1 log("abc")
```

R

```
Error in `log()``:  
! non-numeric argument to mathematical function
```

```
1 sum("abc")
```

R

```
Error in `sum()``:  
! invalid 'type' (character) of argument
```

```
1 undefined_var
```

R

```
Error:  
! object 'undefined_var' not found
```

```
1 "abc" + 1
```

py

```
TypeError: can only concatenate str (not "int") to str
```

```
1 int("abc")
```

py

```
ValueError: invalid literal for int() with base 10: 'ab
```

```
1 [1, 2, 3][5]
```

py

```
IndexError: list index out of range
```

```
1 undefined_var
```

py

```
NameError: name 'undefined_var' is not defined
```

See the Python docs for the full [exception hierarchy](#). Classed conditions are possible in R (e.g. via `rlang::abort()` /

Handling errors

R's `try()` evaluates an expression and, instead of halting, returns a `try-error` object if an error occurs. Python's `try / except` block runs the `except` code only if a matching exception was raised in the `try` body.

```
1 x = try(log("a"), silent = TRUE)
2 class(x)
```

R

```
[1] "try-error"
```

```
1 cat(x)
```

R

```
Error in log("a") : non-numeric argument to
```

```
1 inherits(x, "try-error")
```

R

```
[1] TRUE
```

```
1 try:
2     x = math.log("a")
3 except TypeError as e:
4     print("Caught:", e)
5     x = math.nan
```

py

```
Caught: must be real number, not str
```

```
1 x
```

py

```
nan
```

Exercise 1

Without running the code, what do you expect the output (or error) to be for each of the listed values of `x`?

```
1 if (x > 10 || x < -10) {  
2   stop("Input too big")  
3 } else if (x %in% c(2, 3, 5, 7)) {  
4   cat("Input is prime!\n")  
5 } else if (x %% 2 == 0) {  
6   cat("Input is even!\n")  
7 } else if (x %% 2 == 1) {  
8   cat("Input is odd!\n")  
9 }
```

R

```
1 x = 1  
2 x = 3  
3 x = -1  
4 x = -3  
5 x = 1:2  
6 x = "0"  
7 x = "zero"
```

R

```
1 if x > 10 or x < -10:  
2   raise ValueError("Input too big")  
3 elif x in [2, 3, 5, 7]:  
4   print("Input is prime!")  
5 elif x % 2 == 0:  
6   print("Input is even!")  
7 elif x % 2 == 1:  
8   print("Input is odd!")
```

py

```
1 x = 1  
2 x = 3  
3 x = -1  
4 x = -3  
5 x = [1, 2]  
6 x = "0"  
7 x = 2.5
```

py

Loops

for loops

R's `for` iterates over the elements of a vector (or list), while Python's `for` iterates over the elements of any *iterable* object (lists, tuples, strings, ranges, dictionaries, files, ...).

```
1 for (w in c("Hello", "world!")) {  
2   cat(w, ":", nchar(w), "\n")  
3 }
```

R

```
Hello : 5  
world! : 6
```

```
1 total = 0  
2 for (v in c(1, 2, 3, 4)) {  
3   total = total + v  
4 }  
5 total
```

R

```
[1] 10
```

```
1 for w in ["Hello", "world!"]:  
2   print(w, ":", len(w))
```

py

```
Hello : 5  
world! : 6
```

```
1 total = 0  
2 for v in (1, 2, 3, 4):  
3   total += v  
4 total
```

py

```
10
```

Integer sequences

Loops over indices need a sequence of integers - R has `:`, `seq()`, `seq_len()`, and `seq_along()`, while Python has `range()`.

```
1 1:5
```

R

```
[1] 1 2 3 4 5
```

```
1 seq(1, 10, by = 3)
```

R

```
[1] 1 4 7 10
```

```
1 seq_len(3)
```

R

```
[1] 1 2 3
```

```
1 seq_along(c("a", "b", "c"))
```

R

```
[1] 1 2 3
```

```
1 range(5)
```

py

```
range(0, 5)
```

```
1 list(range(5))
```

py

```
[0, 1, 2, 3, 4]
```

```
1 list(range(1, 11, 3))
```

py

```
[1, 4, 7, 10]
```

```
1 list(range(5, 0, -1))
```

py

```
[5, 4, 3, 2, 1]
```

Looping over indices

The R idiom is `seq_along(x)`. Python's equivalent is `range(len(x))`, but `enumerate()` is preferred as it yields the index *and* the value together (as a tuple, which is unpacked into `i` and `v` below).

```
1 x = c("a", "b", "c")
2 for (i in seq_along(x)) {
3   cat(i, x[i], "\n")
4 }
```

```
1 a
2 b
3 c
```

```
1 x = ["a", "b", "c"]
2 for i in range(len(x)):
3   print(i, x[i])
```

```
0 a
1 b
2 c
```

```
1 for i, v in enumerate(x):
2   print(i, v)
```

```
0 a
1 b
2 c
```

Avoid `1:length(x)`

The common R idiom `1:length(x)` fails for empty vectors since `1:0` is `c(1, 0)` - use `seq_along()` or `seq_len()` instead. Python's `range(len(x))` is safe since `range(0)` is empty.

```
1 x = integer()  
2 1:length(x)
```

R

```
[1] 1 0
```

```
1 seq_along(x)
```

R

```
integer(0)
```

```
1 for (i in 1:length(x)) print(i)
```

R

```
[1] 1
```

```
[1] 0
```

```
1 for (i in seq_along(x)) print(i)
```

R

```
1 x = []  
2 list(range(len(x)))
```

py

```
[]
```

```
1 for i in range(len(x)): print(i)
```

py

Multiple sequences

Python's `zip()` iterates over multiple sequences together, stopping at the shortest. R has no direct equivalent - index with `seq_along()` instead (though vectorization usually makes this unnecessary).

```
1 x = c(1, 2, 3)
2 y = c("a", "b", "c")
3 for (i in seq_along(x)) {
4   cat(x[i], y[i], "\n")
5 }
```

```
1 a
2 b
3 c
```

```
1 x = [1, 2, 3]
2 y = ["a", "b", "c"]
3 for a, b in zip(x, y):
4   print(a, b)
```

```
1 a
2 b
3 c
```

```
1 list(zip([1, 2, 3, 4], "ab"))
```

```
[(1, 'a'), (2, 'b')]
```

`zip()` returns a lazy iterator (hence the `list()` to see its contents). See `itertools.zip_longest()` to iterate until the

while loops

Repeat the body as long as the condition is **TRUE** / **truthy** - the condition is checked before each iteration, so the body may never run.

```
1 i = 1
2 while (i < 100) {
3   i = i * 2
4 }
5 i
```

R

```
1 i = 1
2 while i < 100:
3     i *= 2
4 i
```

py

128

[1] 128

R also has **repeat**, which loops forever until a **break** - the Python idiom for this is **while True:**.

```
1 i = 1
2 repeat {
3   i = i * 2
4   if (i >= 100) break
5 }
6 i
```

R

```
1 i = 1
2 while True:
3     i *= 2
4     if i >= 100:
5         break
6 i
```

py

128

[1] 128

break and next / continue

break exits the (innermost) loop entirely, while **next** in R and **continue** in Python skip the rest of the current iteration.

```
1 for (i in 1:10) {  
2   if (i %% 3 == 0) next  
3   cat(i, "")  
4 }
```

R

1 2 4 5 7 8 10

```
1 for i in range(1, 11):  
2   if i % 3 == 0:  
3       continue  
4   print(i, end=" ")
```

py

1 2 4 5 7 8 10

```
1 for (i in 1:10) {  
2   if (i %% 3 == 0) break  
3   cat(i, "")  
4 }
```

R

1 2

```
1 for i in range(1, 11):  
2   if i % 3 == 0:  
3       break  
4   print(i, end=" ")
```

py

1 2

Loop `else` and `pass`

Two Python-only constructs - loops can have an `else` clause that runs when the loop finishes *without* a `break`, and `pass` is a no-op placeholder for where a statement is syntactically required.

```
1 for n in range(2, 10):
2     for x in range(2, n):
3         if n % x == 0:
4             print(n, "=", x, "*", n // x)
5             break
6     else:
7         print(n, "is prime")
```

py

```
1 x = -3
2 if x < 0:
3     pass
4 elif x % 2 == 0:
5     print("x is even")
6 else:
7     print("x is odd")
```

py

```
2 is prime
3 is prime
4 = 2 * 2
5 is prime
6 = 2 * 3
7 is prime
8 = 2 * 4
9 = 3 * 3
```

Loop `else` example from the [Python Tutorial](#). R has no loop `else`, and an empty block `{ }` (or `NULL`) serves as a

Building up results

```
1 res = c()  
2 for (i in 1:5) {  
3   res = c(res, i^2)  
4 }  
5 res
```

R

```
[1] 1 4 9 16 25
```

```
1 res = []  
2 for i in range(1, 6):  
3   res.append(i**2)  
4 res
```

py

```
[1, 4, 9, 16, 25]
```

Growing an R vector with `c()` copies it every iteration (quadratic runtime), so for longer loops preallocate (`numeric(n)`, `character(n)`, `vector("list", n)`) and assign by index. Python's `list.append()` is amortized constant time, so appending is idiomatic.

```
1 res = numeric(5)  
2 for (i in seq_along(res)) {  
3   res[i] = i^2  
4 }  
5 res
```

R

```
[1] 1 4 9 16 25
```

In practice most loops like these are replaced by vectorized operations, functional tools (`lapply()`, `purrr::map()`), or

Exercise 2

To the right are vectors containing all prime numbers between 2 and 100 and some values `x` we would like to check for primality.

Using *nested* loops, write code in both R and Python that prints only the values of `x` that are *not* prime - without using subsetting, `%in%`, or `in`.

In Python, try using the loop `else` clause; in R you will need a flag variable.

```
1 primes = c( 2, 3, 5, 7, 11, 13, 17, 19, 23,  
2           29, 31, 37, 41, 43, 47, 53, 59, 61,  
3           67, 71, 73, 79, 83, 89, 97)  
4 x = c(3, 4, 12, 19, 23, 51, 61, 63, 78)
```

```
1 primes = [ 2, 3, 5, 7, 11, 13, 17, 19, 23,  
2           29, 31, 37, 41, 43, 47, 53, 59, 61,  
3           67, 71, 73, 79, 83, 89, 97]  
4 x = [3, 4, 12, 19, 23, 51, 61, 63, 78]
```

Comparing R & Python

Control flow summary

Construct	R	Python
conditional	<code>if / else if / else</code>	<code>if / elif / else</code>
conditional expression	<code>if (cond) a else b</code>	<code>a if cond else b</code>
vectorized conditional	<code>ifelse()</code>	<code>comprehension, np.where()</code>
multi-way branch	<code>switch()</code>	<code>match / case</code>
for loop	<code>for (x in vec) {}</code>	<code>for x in iterable:</code>
while loop	<code>while (cond) {}</code>	<code>while cond:</code>
infinite loop	<code>repeat {}</code>	<code>while True:</code>
skip iteration	<code>next</code>	<code>continue</code>
exit loop	<code>break</code>	<code>break</code>
integer sequences	<code>:, seq_len(), seq_along()</code>	<code>range()</code>
index + value	<code>seq_along() + x[i]</code>	<code>enumerate()</code>
multiple sequences	<code>seq_along() + indexing</code>	<code>zip()</code>
reduce logicals	<code>any(), all()</code>	<code>any(), all()</code>
blocks	<code>{ }</code>	<code>: + indentation</code>

Error handling summary

Concept	R	Python
message	<code>message()</code>	<code>print()</code> , <code>logging</code>
warning	<code>warning()</code>	<code>warnings.warn()</code>
error	<code>stop()</code>	<code>raise SomeError()</code>
assertion	<code>stopifnot()</code>	<code>assert</code>
catch	<code>try()</code> , <code>tryCatch()</code>	<code>try / except</code>
always run	<code>tryCatch(finally =), on.exit()</code>	<code>finally</code>
error object	<code>condition (simpleError)</code>	exception (subclass of <code>Exception</code>)
error message	<code>conditionMessage(e)</code>	<code>str(e)</code>

Takeaways

- R's comparison and logical operators (`==`, `&`, `|`) are vectorized; `&&` / `||` in R and `and` / `or` in Python are scalar and short-circuit.
- `if` in R needs exactly one `TRUE` / `FALSE` - use `any()`, `all()`, or `ifelse()` for vectors. Python's `if` tests the *truthiness* of any object.
- Loop syntax is nearly identical - the differences are blocks (`{}` vs indentation), `seq_along()` vs `range()` / `enumerate()`, and `next` vs `continue`.
- Errors are *conditions* in R (`stop()`, `tryCatch()`) and typed *exception* objects in Python (`raise`, `try` / `except`).